

tf.stop_gradient Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/stop_gradient

Stops gradient computation.

Function Signatures

```
tf.stop_gradient( input, name=None )  
  
def softmax(x): numerator = tf.exp(x) denominator =  
tf.reduce_sum(numerator) return numerator / denominator  
  
def stable_softmax(x): z = x - tf.reduce_max(x) numerator =  
tf.exp(z) denominator = tf.reduce_sum(numerator) return  
numerator / denominator  
  
tf.reduce_max(x)
```

Code Examples

```
tf.stop_gradient(  
input, name=None  
)
```

```
tf.stop_gradient(  
input, name=None  
)
```

```
def softmax(x):  
    numerator = tf.exp(x)  
    denominator = tf.reduce_sum(numerator)  
    return numerator / denominator
```

```
def softmax(x):  
    numerator = tf.exp(x)  
    denominator = tf.reduce_sum(numerator)  
    return numerator / denominator
```

```
def stable_softmax(x):  
    z = x - tf.reduce_max(x)  
    numerator = tf.exp(z)  
    denominator = tf.reduce_sum(numerator)  
    return numerator / denominator
```

```
def stable_softmax(x):  
    z = x - tf.reduce_max(x)  
    numerator = tf.exp(z)  
    denominator = tf.reduce_sum(numerator)  
    return numerator / denominator
```

```
def stable_softmax(x):  
    z = x - tf.stop_gradient(tf.reduce_max(x))  
    numerator = tf.exp(z)  
    denominator = tf.reduce_sum(numerator)  
    return numerator / denominator
```

```
def stable_softmax(x):  
    z = x - tf.stop_gradient(tf.reduce_max(x))  
    numerator = tf.exp(z)  
    denominator = tf.reduce_sum(numerator)  
    return numerator / denominator
```

Documentation

Stops gradient computation.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.stop_gradient`

Used in the notebooks

- Advanced automatic differentiation

When executed in a graph, this op outputs its input tensor as-is.

When building ops to compute gradients, this op prevents the contribution of its inputs to be taken into account. Normally, the gradient generator adds ops to a graph to compute the derivatives of a specified 'loss' by recursively finding out inputs that contributed to its computation. If you insert this op in the graph its inputs are masked from the gradient generator. They are not taken into account for computing gradients.

This is useful any time you want to compute a value with TensorFlow but need to pretend that the value was a constant. For example, the softmax function for a vector x can be written as

This however is susceptible to overflow if the values in x are large. An alternative more stable way is to subtract the maximum of x from each of the values.

However, when we backprop through the softmax to x , we don't want to backprop through the `tf.reduce_max(x)` (if the max values are not unique then the gradient could flow to the wrong input) calculation and treat that as a constant. Therefore, we should write this out as

Some other examples include:

- The EM algorithm where the M-step should not involve backpropagation

through the output of the E-step.

- Contrastive divergence training of Boltzmann machines where, when

differentiating the energy function, the training must not backpropagate through the graph that generated the samples from the model.

- Adversarial training, where no backprop should happen through the adversarial

example generation process.

Returns